

Ein Dateisystem ist eine Möglichkeit Dateien und Ordner zu erstellen, zu löschen, zu speichern und zu manipulieren. Hier geht es vor allem um die technische Sicht, sprich: **Es geht darum, wo etwas gespeichert wird, und was gespeichert wird.

Physische Speichermedien

Festplatten

Festplatten sind physische Speichermedien, welche es ermöglichen, eine große bis sehr große Menge an Daten zu speichern und wieder zu verändern. Hierbei ist die Speichergröße, Geschwindigkeit, Anzahl der Lese- und Schreibvorgänge sowie deren Logik abhängig von der Bauweise der Festplatten.

Im folgenden werden die beiden häufigsten Bauformen betrachtet und verglichen:

HDDs - Hard Disk Drives

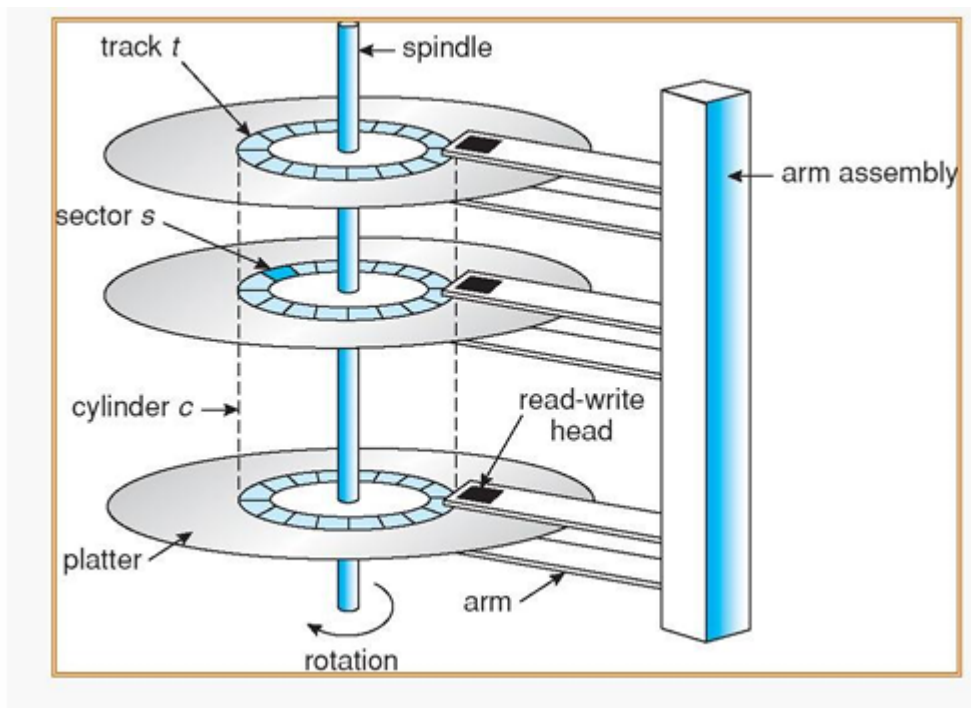
Key-Points

- HDDs bestehen aus rotierenden Platten und Lese/Schreibarmen
- Die Platten sind in Sektoren zu 512 Bytes aufgeteilt
- Dateien werden hintereinander gespeichert, um möglichst schnell zu sein.
- Für große Dateien werden Sektoren zu Clustern zusammengefasst um noch effizienter zu sein
- Nicht volle Sektoren / Speicher verschwenden ungefüllten Speicherplatz

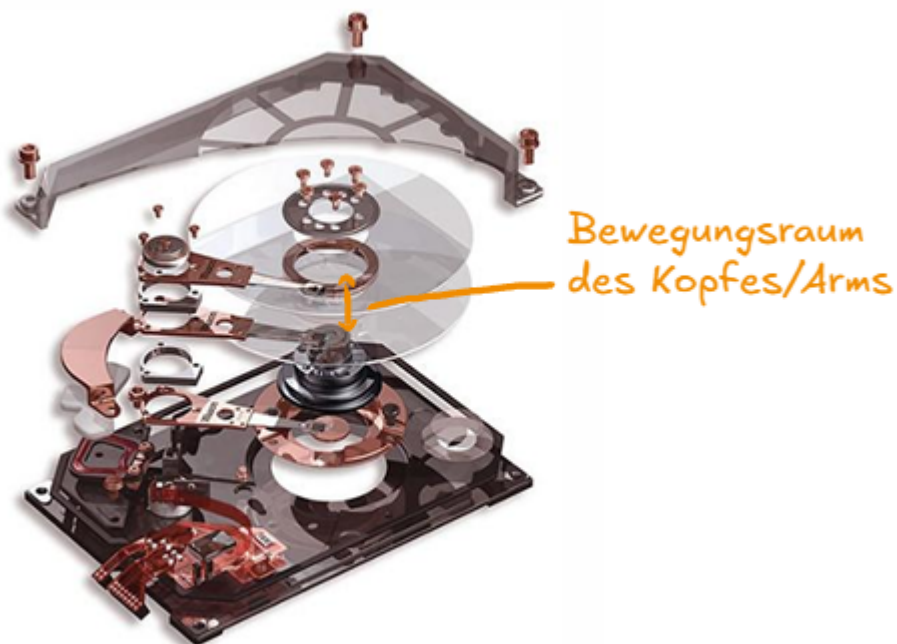
Vorteile vs. Nachteile

- Vorteile:
 - Robust
 - billiger (heute nicht mehr allzu wahr)
 - altersresistent (Daten gehen nicht über Zeit verloren)
- Nachteile:
 - Viel langsamer als zB HDDs
 - Bewegliche Teile gefährdet (zB durch fallen lassen oder zeitlichen decay)

HDDs bestehen aus Speicherplatten (Hard-Disks), auf welche Dateien geschrieben und wieder gelesen werden können. Dies geschieht über einen Schreib- und Lesekopf, welcher an einem Arm montiert ist. Ist die HDD an, drehen sich die Scheiben, und der Arm kann dann die Daten auslesen.



Die Rotation der Scheiben sorgt dafür, dass der Arm auf 360° der Scheibe Zugriff hat. Der Arm selbst kann sich auch bewegen, und zwar nach außen oder innen. Dadurch kann er jede Tiefe der Scheibe erreichen (siehe Bild)



Gespeichert werden die Daten in **Sektoren** von **512 Bytes** gespeichert. Ist eine Datei größer als 512 Byte, wird gewöhnlich der Rest solange in den folgenden Sektor geschrieben, bis die Datei komplett abgelegt ist. Das bedeutet, **Dateien werden hintereinander gespeichert**. Das wird gemacht, um die **Bewegung des Armes so gering wie möglich zu halten**, da diese Zeit in Anspruch nimmt.

Blöcke und Cluster

Für Dateisysteme mit großen Dateien würde eine Sektorgröße von mehr als 512 Bytes Sinn ergeben. Da jeder Sektor einzeln gelesen und beschrieben wird, würden größere Sektoren auch weniger Vorgänge bedeuten. Daher gibt es die Möglichkeit, Sektoren zu sogenannten **Clustern bzw. Blocks** zu verbinden. Das hat allerdings den Nachteil, dass dadurch potentiell Speicherplatz verloren gehen kann.

Warum?

Sektoren, welche nicht komplett gefüllt werden, können nicht mehr aufgefüllt werden, weil der Kopf immer nur am Beginn eines Sektors zu schreiben beginnt (Ein Sektor ist quasi die kleinste Einheit auf der HDD, sie kennt nichts kleineres). Bleibt ein Teil des Sektors leer, ist dieser Speicherplatz vergeudet. Je größer nun die Sektoren, desto mehr Speicherplatz kann offen bleiben. Daher gilt:

Kleine Files: Kleinere Sektoren (dafür langsamer)

Große Files: Größere Sektoren (dafür potentiell Speicherplatzverlust)

SSDs Solid State Drives

Key-Takeaways

- SSDs bestehen aus NAND Flashspeicher sowie einem Controller
- Die Speicherzellen werden in verschiedene Größen zusammengelegt, am relevantesten sind Pages (4Kbyte) und Blöcke (512 KByte)
- Pages können im Live, Dead oder Free Zustand sein
- SSD Speicher hält nur einer endlichen Menge an Lese- und Schreibvorgängen stand. Damit nicht immer die selben Zellen beansprucht werden, wird Wear Levelling verwendet

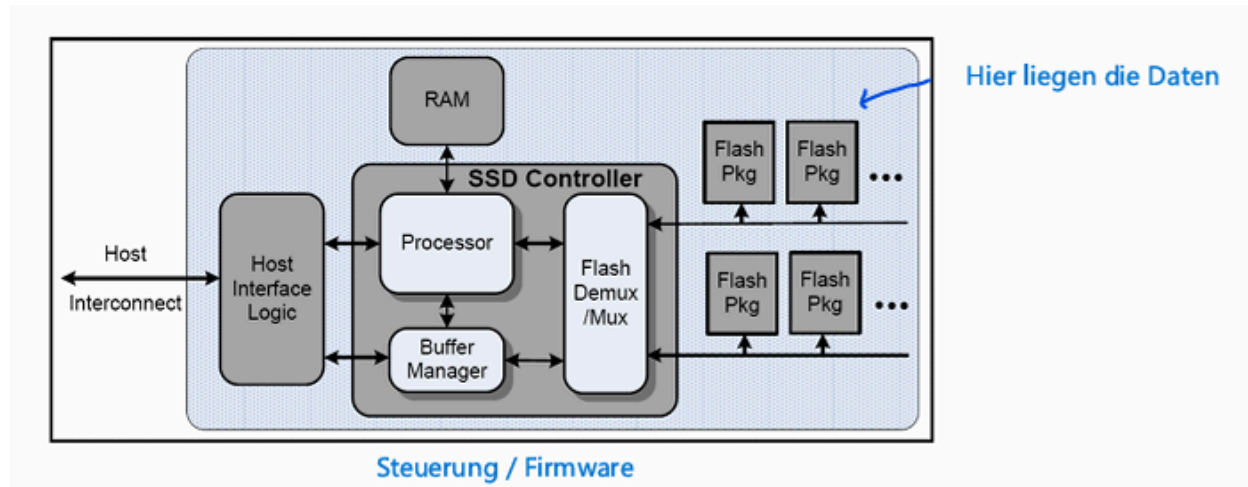
Vorteile vs. Nachteile

- Vorteile:
 - Extrem schnell
 - Keine beweglichen Teile
 - Formfaktor viel kleiner als HDD
 - Speicher inzwischen $\geq$ HDDs
- Nachteile:
 - Endliche Schreibvorgänge
 - Daten verschwinden wenn lange kein Strom angeschlossen wird

Eine SSD ist ein physisches Speichermedium, welches die gleichen Aufgaben erfüllt wie eine HDD und auch ähnlich angesprochen werden kann. Sie sind quasi der Nachfolger der

HDDs.

Anders als HDDs verfügen SSDs nicht über Speicherscheiben oder andere bewegliche Teile wie Lese- und Schreibköpfe. Sie bestehen aus einem Prozessor, welcher über ein Interface mit dem Computer kommuniziert, sowie einer großen Menge an NAND- Flashspeicherzellen (siehe 1. Semester EIR), sowie einem kleinen Cache. Die Datenlese- und Schreibvorgänge passieren rein elektrisch über das Induzieren von Elektronen in Elektronenfallen und das Auslesen dieser durch Messen von Spannungspegeln.



Mehr dazu siehe:

[How do SSDs work \(short\)](#)

[How do SSDs work \(in depth\)](#)

[How does NAND Flash work?](#)

Da jede Speicherzelle einzeln anzusteuern wieder extrem langsam und ineffizient wäre, werden die Zellen analog zur HDD zusammengefasst. Die Zusammenfassung geschieht nach folgendem Schema:

- 4096 Zellen -> 1 Page (4 KByte)
- 128 Pages -> 1 Block (512 KByte)
- 1024 Blöcke -> 1 Plane (512 MByte)
- 4 Planes -> 1 Die (2 GByte)
- 2 Dies -> 1 Flash Package (4 GByte) (Das, was oben im Bild gezeigt ist)

Page Zustände

Das Beschreiben von SSDs läuft auf der Ebene der Pages, wobei nur ganze Blöcke gelöscht werden können (Analoger Speicherverlust wie bei HDD)

Pages haben verschiedene **Zustände**

- Live Pages

- Funktionierende, beschriebene Pages, können ausgelesen oder freigegeben ("gelöscht") werden
- Dead Pages
 - Pages welche aufgrund von Alter oder Systemversagen nicht mehr beschrieben werden können / dürfen.
- Free Pages
 - Pages welche als frei markiert sind. Diese können mit neuen Daten beschrieben werden. Geschieht dies, werden sie zu Live Pages.

Da Pages eine endliche Menge an Schreibvorgängen aushalten, verwendet der SSD Controller **Wear Levelling** um die Zellen immer gleichmäßig zu beschreiben und freizugeben. Er priorisiert weniger verwendete Pages und schichtet auch Daten regelmäßig um, damit die Chance von Datenverlust reduziert wird.

Um Dateien überhaupt finden zu können, verwendet der Controller Blockadressen. Das Dateisystems des Computers selbst hat keine Ahnung mehr, wo die Dateien auf der SSD genau liegen.

Logische Sicht eines Dateisystems

Key Takeaways

- Dateisysteme verbinden das OS mit Partitionen am Datenträger und haben die Aufgabe, viele Daten zu speichern und später wieder schnell abzurufen
- Der Begriff Dateisystem ist mehrdeutig und kann auch nur für die Dateistruktur verwendet werden
- Die Daten müssen dauerhaft (persistent) abgelegt sein und parallel verwendbar sein
- Dateisysteme bestehen aus **Dateien, Verzeichnissen und (W) Laufwerken**

Ein Dateisystem ist die **Schnittstelle zwischen dem Betriebssystem und den Partitionen auf den Datenträgern**. Es hat die Aufgabe, Dateien einfach ablegbar und später wieder schnell findbar zu machen.

Es gibt verschiedene Dateisysteme, welche jeweils eine eigene **Partition (Speicherbereich auf einer Festplatte)** benötigen um arbeiten zu können.

Der Begriff Dateisystem selbst kann sich auf die Partition, sowie auf die Struktur beziehen. Das Dateisystem ist für den Nutzer die Vereinfachung und Darstellung des Datenspeicherprozesses auf einer Festplatte.

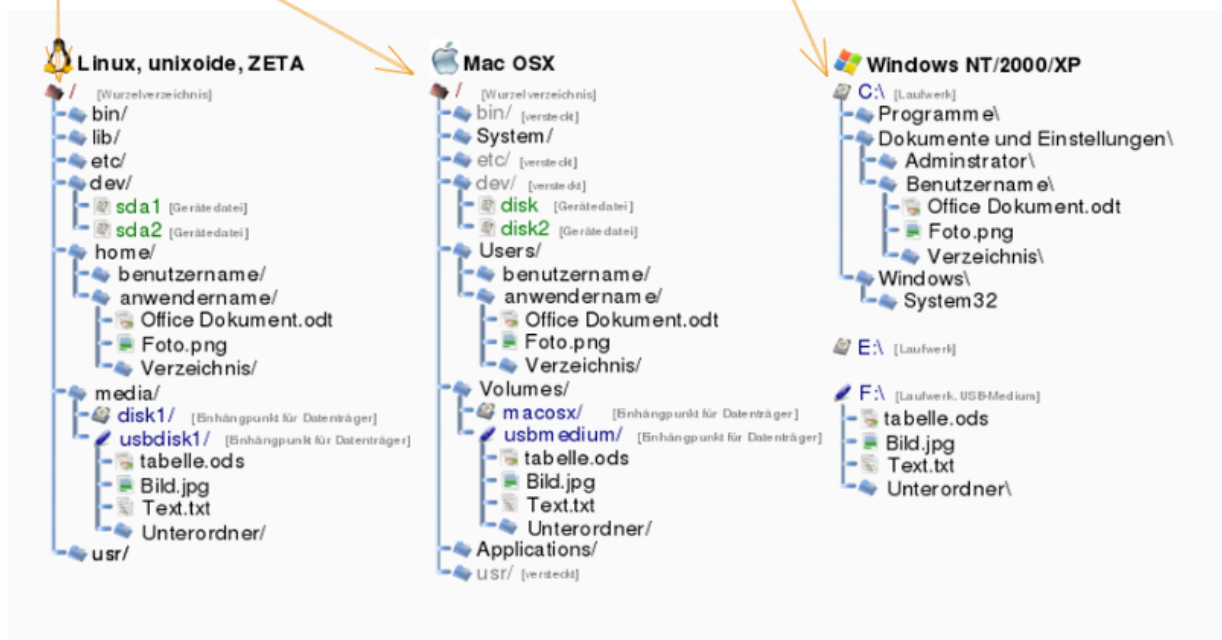
Aufgaben

Ein Dateisystem verfolgt folgende Kernaufgaben:

- Einteilung von Daten in Datenstrukturen
 - Dateien
 - Verzeichnisse
 - Laufwerke (Windows only)
- Persistente (dauerhafte) Speicherung großer Datenmengen
- Zugriff auf Daten durch **mehrere Prozesse parallel**
- Berechtigungssystematik (nicht jeder soll alles machen dürfen)

Root Directory
"Wurzelverzeichnis"

C Laufwerk
(quasi das Root von Windows)



Dateien vs. Verzeichnis

Key Takeaways

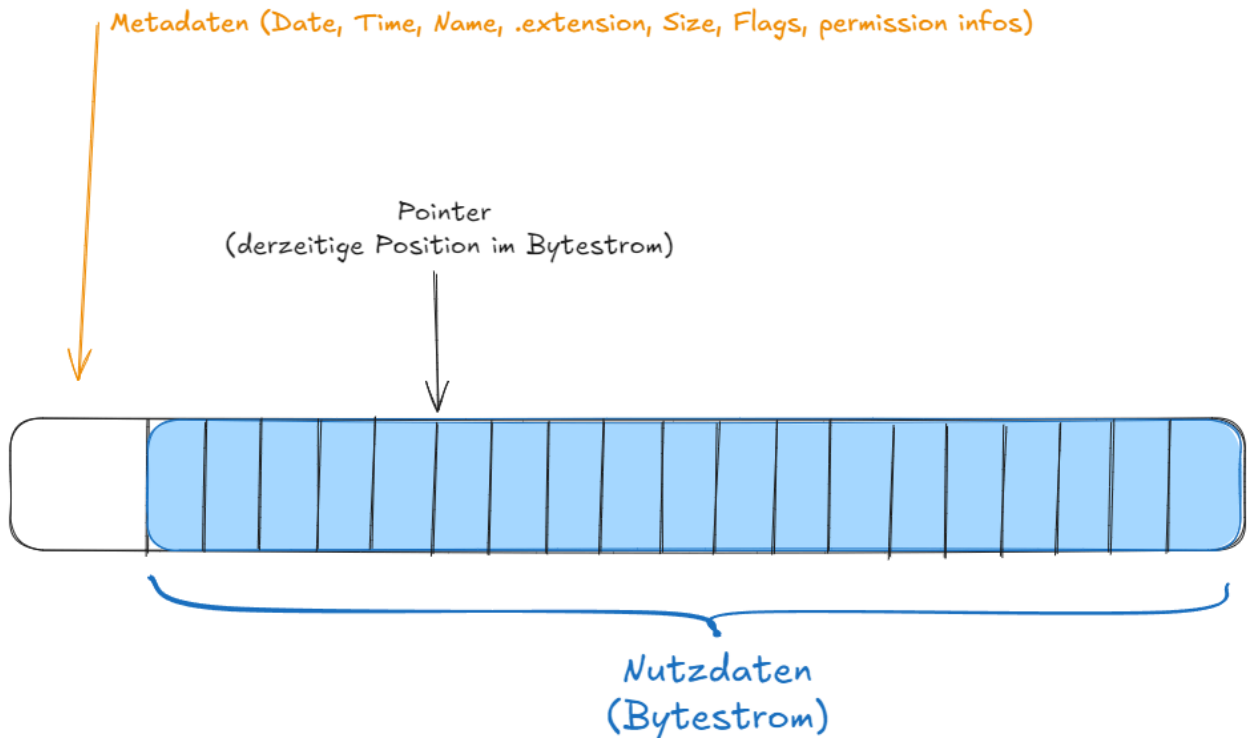
- Dateien sind die kleinste Einheit des Systems, Verzeichnisse eine Überstruktur
- Dateien halten die eigentlichen Nutzdaten, Verzeichnisse ermöglichen logisches Sortieren der Dateien
- Dateien bestehen aus Metadaten und Nutzdaten
- Es gibt verschiedene Dateiformate für verschiedene Anwendungsarten, meist für Hardwarekommunikation
- Je nach Betriebssystem sind Dateien und Verzeichnisse unterschiedlich implementiert

Dateien sind die kleinste Komponente eines Dateisystems. Sie speichern die eigentlichen Nutzdaten. Verzeichnisse sind eine übergeordnete Struktur, welche die Sortierung und Zuordnung von Dateien ermöglicht.

Dateien - Logische Sicht

Dateien bestehen prinzipiell aus zwei Teilen:

- Metadaten - Diese geben Informationen **über** die Datei an, zB Name und Erstellungszeit
- Nutzdaten - Diese geben die Informationen **in** der Datei an, alias alles was in der Datei gespeichert werden soll.



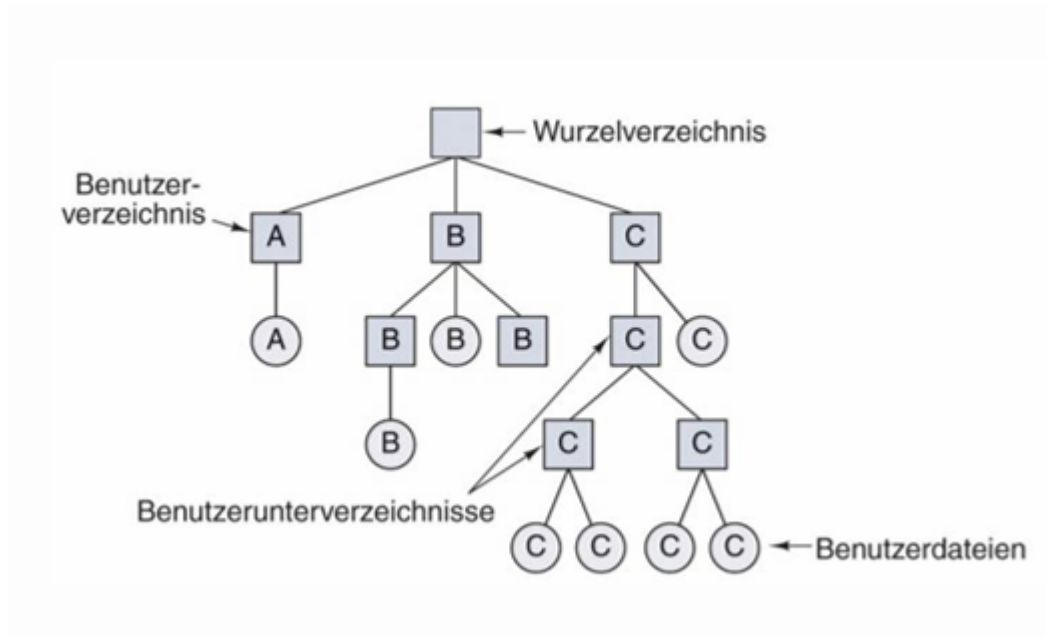
Weiters gibt es für verschiedene Anwendungen unterschiedliche Dateiarten (Achtung, hier sind nicht die unterschiedlichen Extensions wie zB. .png, .pdf, oder .exe gemeint, sondern Dateiarten welche einen komplett anderen Ansatz der Datenspeicherung verfolgen!)

- Reguläre Dateien (siehe oben)
- Verzeichnisse (LINUX - UNIX spezifisch)
- Blockdateien - (Zur Kommunikation mit Blockbasierten Geräten zB SSDs)
- Zeichendateien - (Zur Kommunikation mit Zeichenbasierten Geräten zB meiste Hardware, alles über USB)

Verzeichnisse - Logische Sicht

Verzeichnisse sind eine Überstruktur, welche es ermöglicht Dateien in für den Nutzer logische Gruppen zu sortieren und diese sortiert abzulegen. Ein Verzeichniss kann Unterverzeichnisse enthalten, welche wieder Unterverzeichnisse enthalten können etc. Ein

Oberverzeichnis kann aber kein Unterverzeichnis eines Unterverzeichnisses des Oberverzeichnisses sein! (Baumstruktur, nicht "nur" gerichteter Graph)



Je nach Betriebssystem sind Verzeichnisse intern anders aufgebaut, funktionieren tun sie jedoch weitestgehend gleich.

Partitionen, Partitionieren und Booten eines Computers

Partitionen - Key Takeaways

- Partitionen sind durch ein Programm festgelegte Abschnitte auf der Festplatte
- Partitionen dienen der Abtrennung von Datei- und Betriebssystemen
- Partitionierung ermöglicht mehrere Systeme auf einem physischen Speicher
- Die Menge an Partitionen hängt von der Firmware ab, wobei BIOS einschränkender ist als UEFI

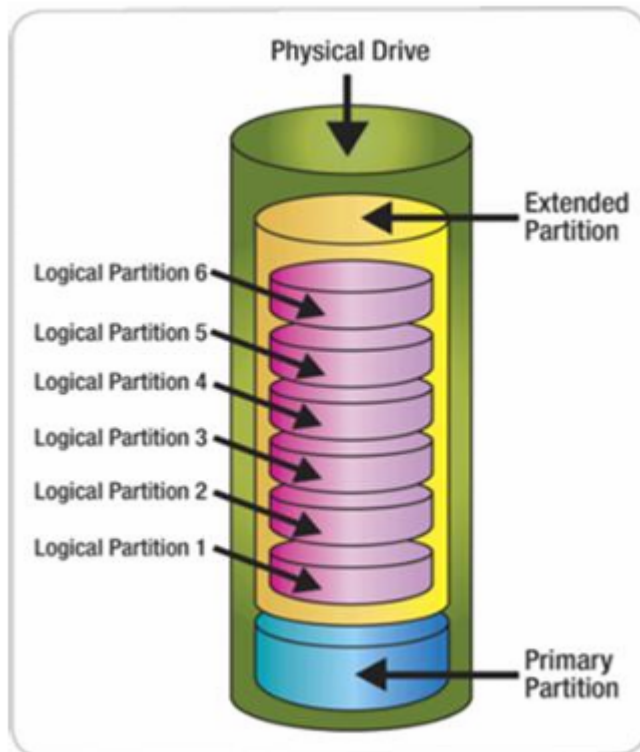
Partitionen sind durch ein Programm festgelegte Bereiche auf einer Festplatte. Je nach Art der Partition wird auf diesen ein Betriebssystem (Boot-Partition) oder ein Dateisystem gespeichert.

Partitionierung wird durchgeführt, damit **kein System im Bereich eines anderen Systems Daten verändern kann**. Ohne Partitionen könnte zB ein Dateisystem versehentlich Boot-Routinen überschreiben.

Das Partitionieren erlaubt auch **mehrere Betriebs- und Dateisysteme auf einer Festplatte zu speichern**.

Die Partition, welche die Informationen für den Bootvorgang beinhaltet wird als **Primäre Partition** bezeichnet. Alle anderen werden als **Erweiterte Partition** bezeichnet.

Die erweiterte Partition dient als Container, in welchem weitere "logische Partitionen" angelegt werden können. Dies wurde unter BIOS gemacht, um das Limit von 4 Partitionen zu umgehen.



Wie eine Partition erstellt werden kann und wie viele es von welcher Partition geben kann, wird durch die Firmware angegeben. Hierbei erlauben ältere Firmwares (BIOS/MBR) um einiges weniger Freiraum als neue (UEFI/GPT)

- BIOS/MBR -> 4 primary oder 3 primary und 1 extended partition
- UEFI/GPT -> 128 primary (does not need extended Partitions anymore)

Genauerer hierzu: [Primäre, Erweiterte, Logische Partitionen - Unterschied](#)

Booten eines Computers

Booten beschreibt den Startvorgang eines Computers. Das Booten wird durch die Firmware eingeleitet, welche auf einer sogenannten **Boot-ROM** (Read Only Memory) auf dem Motherboard gespeichert ist.

Die Boot-Rom ist für folgende Aufgaben zuständig:

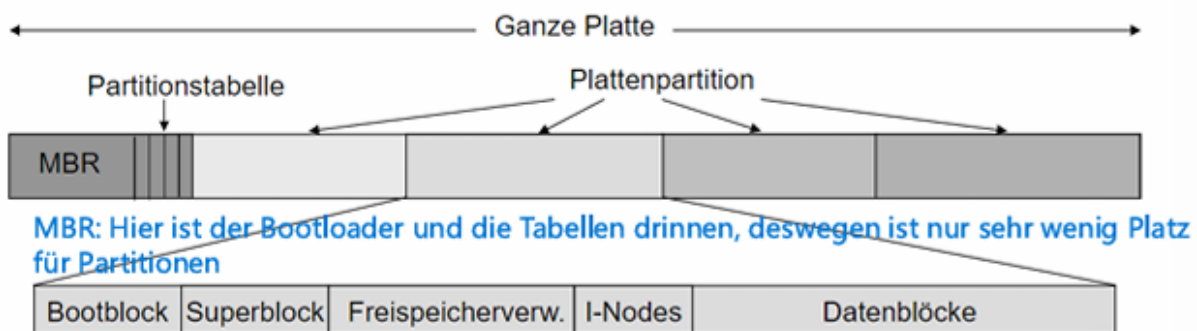
- Hardware initialisieren
- Services für OS bereitstellen
- Gerät bestimmen, von welchem gebootet werden soll (Einstellbar im Bootmenü)
die bekanntesten Boot-Programme sind
- BIOS (Basic Input/Output System)
 - Verwendet den Master Boot Record (MBR)
- UEFI (Unified Extensible Firmware Interface)

- Verwendet GUID Partition Table (GPT)

Layout von Festplatte, Partition und Dateisystem

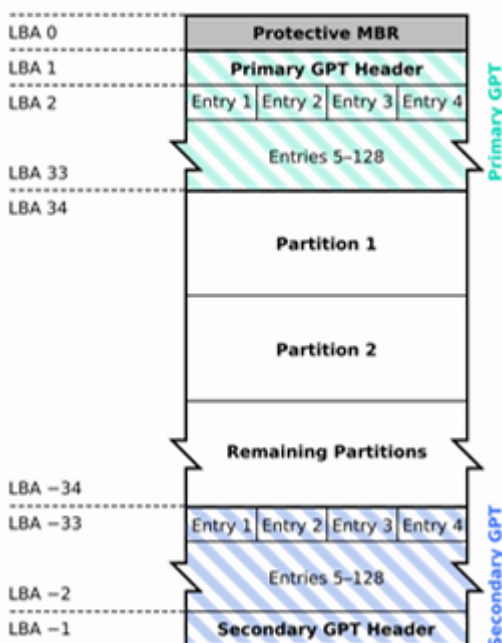
Je nach verwendeter Firmware ist das Layout der Festplatte und der darauf gespeicherten Partitionen unterschiedlich. Für beide jedoch gilt, dass die relevanten Tables (MBR und GPT) immer ganz vorne abgelegt werden.

Da die Partitionstabelle, welche Beginn, Ende und Infos über jede Partition enthält, im MBR extrem klein ist, kann nur ein Maximum von 4 Partitionen angelegt werden. Diese Tabellengröße ist allerdings arbitrary und hätte anders gewählt werden können, wäre man bereit gewesen dafür mehr Speicherplatz zu nehmen.



Im Vergleich zu MBR verfügt GPT über ein sekundäres Backup, welches am Ende der Festplatte angelegt wird. Das hilft, falls etwas im Speicher schief geht, da nicht sofort die gesamte Platte dadurch unbrauchbar wird.

GUID Partition Table Scheme



Genauere Abläufe des Bootens durch GPT findet sich im **Foliensatz Dateisysteme S.27**

Dateisysteme aus OS-Sicht

Key Takeaways

- Ein Dateisystem ist so ausgelegt, dass es mit der Geschwindigkeit und dem Speicherplatz möglichst effizient umgeht
- Durch das möglichst Nebeneinanderspeichern von Daten sollen Fragmentierung und Speicherverschwendung vermieden werden.
- Das OS stellt Systemcalls für Dateien und Verzeichnisse bereit, um diese zu manipulieren
- Dateisysteme können auch Zusätzliches wie automatische Sicherung, Verschlüsselung und Datenkomprimierung

Ein Dateisystem muss mit seiner Partition möglichst effizient arbeiten. Jegliche Ineffizienz führt logischerweise zu Verzögerungen und einem schlechteren Nutzungserlebnis.

Folgende Grundsätze halten Dateisysteme bei der Partitionsverwaltung zumindest ein:

- Positionierungszeit von Lese- und Schreibkopf wird minimiert (nur für HDDs relevant, wird immer irrelevanter)
- Wird eine Datei gespeichert, werden Blöcke möglichst nahe aneinander abgelegt, um Fragmentierungsprobleme zu vermeiden
- Die Kapazität der Partition soll möglichst ausgenutzt werden (möglichst keine halbvollen Blöcke o. ä.)

Um mit den Dateien und Verzeichnissen interagieren zu können, braucht der Nutzer, bzw. die Programme und Routinen **System Calls**, also Befehle, welche das OS dann stellvertretend ausführt.

System Calls bei Dateien sind beispielsweise:

- create
- delete
- open
- close
- read
- write
- append
- seek
- rename
- ...

bzw. für Verzeichnisse:

- create

- delete
- opendir
- closedir
- rename
- ...

Neben dem puren Erstellen von Dateien kann ein Dateisystem auch einige **Zusatzfunktionen** bieten. Eine sehr wichtige ist hierbei das **Erstellen eines Dateisystems auf einer leeren Partition**.

Je nach Anwendung können Dateisysteme die Daten auch von selbst **komprimieren und/oder verschlüsseln** oder **BackUps erstellen**.

Adressierung

Key Takeaways

- Für das Dateisystem ist die Partition quasi ein Array aus Speicherblöcken
- Diese werden vom OS/Dateisystem über sogenannte Logische Blockadressen (LBAs) adressiert
- Durch die Abstraktion der physischen Adressen auf LBAs kann ein System mit vielen Festplattenarten arbeiten
- Das Dateisystem selbst ist in der Partition gespeichert und braucht Speicherplatz.

Während wir eine Partition als ein Teilbereich einer Festplatte begreifen, sieht das Dateisystem die Partition als **Array aus Speicherblöcken**, welche, wie oben erläutert, **frei oder belegt** sein können.

Hierbei ist die Reihenfolge der Speicherblöcke im Array nicht zwingend die physische Reihenfolge auf der Festplatte.

Jeder einzelne Block verfügt über eine **eigene Blockadresse (LBA - Logical Block Address)**

mit welcher er gezielt angesprochen werden kann (sie ist quasi der Array - Index).

LBAs haben derzeit standardmäßig eine Größe von 48 Bit

! Natürlich muss irgendwo bekannt sein, welche LBA zu welchem physischen Block gehört. Das ist allerdings nicht die Aufgabe des Dateisystems, sondern des Festplattencontrollers. !

Durch die Abstraktion der physischen Adressen in LBAs kann ein Dateisystem für verschiedenste Festplattentypen funktionieren.

Neben den Dateien und Verzeichnissen muss das Dateisystem sich selbst ebenfalls auf der selben Partition ablegen, sprich jede Partition erleidet durch die reine Anwesenheit des Dateisystems einen Verlust an freien Datenblöcken.

(Der Schichten des Dateisystems werden übergangen, genaueres auf S.33 des Foliensatzes)

Layout eines generischen Dateisystems

Key - Points

Ein Dateisystem besteht aus 4 Hauptblöcken

- Systeminformationen
- Inhaltsverzeichnis
- Freispeicherverwaltung
- Nutzdaten

Allgemein besteht ein Dateisystem aus folgenden Bestandteilen:

- Informationen über das DS
 - Dateisystemkennung, **Blockgröße**, Dirty Bit
- Inhaltsverzeichnis für Dateien und Verzeichnisse
 - Dateiattribute und Referenzen auf jede Datei und Verzeichnis (WO liegen die Daten?)
- Freispeicher / Bad Block List
- Datenbereich für Dateien und Verzeichnisse
 - Die eigentlichen belegten und freien Nutzdatenblöcke (WAS steht in den Daten?)



Dirty Bit: Ein Bit, welches als Indikator fungiert, ob ein Systemabsturz während einer relevanten Aktion des Dateisystems passiert ist. Hilft, Inkonsistenzen zu verhindern.

Bad Block List: Liste der zuvor beschriebenen "Bad Blocks", also jenen Speicherblöcken, welche aufgrund des Alters / der vielen Nutzung nicht mehr verwendet werden oder schlichtweg kaputt sind.

Freispeicherverwaltung

Key Takeaways

- Die Freispeicherverwaltung speichert welche Blöcke frei und welche belegt sind
- Dafür kommen Bitmaps oder Listen zum Einsatz
- Bitmaps haben eine konstante Größe und halten ein Minimum an Information
- Listen können kleiner als Bitmaps sein, brauchen potentiell sehr viel mehr Platz, halten dafür aber auch mehr Information

Die Freispeicherverwaltung hat, wie der Name schon sagt, die Aufgabe den leeren, bzw. freien, Speicher zu verwalten. Das bedeutet vor allem im Blick zu haben, welche Speicherblöcke überhaupt frei sind. Hier ist vor allem **Konsistenz** wichtig. Passiert etwas unvorhergesehenes und Blöcke werden trotz Löschung nicht als frei markiert, oder schlimmer, trotz Beschreibens nicht als belegt markiert, führt dies zu Speicher und/oder Datenverlust!

Um die Blöcke zu kategorisieren, gibt es zwei Hauptverfahren:

- Bitmaps
- Verkettete Listen

Bitmaps haben den Vorteil, dass ihre Größe immer gleich ist. Jeder Block wird von genau einem Bit repräsentiert, wobei 1 = voll und 0 = leer ist. Diese Bitmap ist zu jedem Zeitpunkt gleich groß und ist insgesamt recht klein, allerdings ist die Information durch den geringen Speicher pro Block auch sehr beschränkt.

Bitmaps werden von Systemen wie ext4, ntfs, welche das einfachere Handling für den "Speicherplatzverlust" in Kauf nehmen.

Listen haben den Vorteil, dass sie vor allem auf Systemen mit vielen freien Blöcken kleiner als die Bitmap ist. Zusätzlich zu einem Pointer auf die belegten Blöcke könnten noch weitere Informationen gespeichert werden, allerdings hat das den Nachteil, dass der Speicherplatz für diese Listen bei wachsenden Systemen exponentiell größer wird.

Inhaltsverzeichnis der Dateien

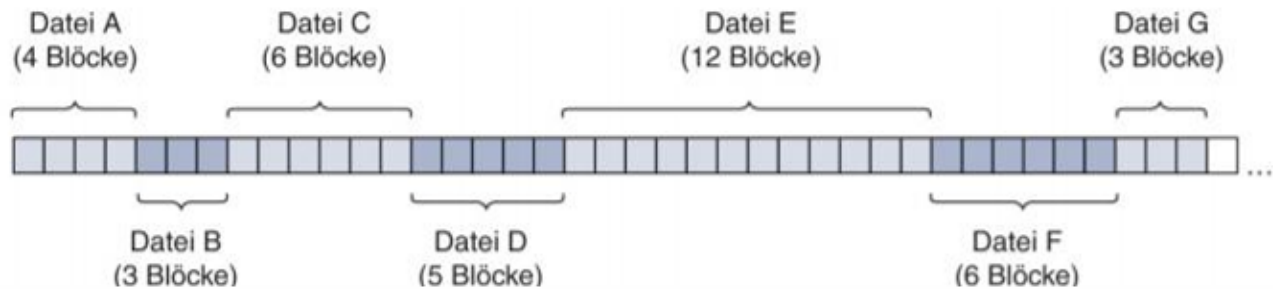
Dateien werden auf der Festplatte in den oben erläuterten Blöcken abgelegt. Das Problem ist, dass Dateiblöcke nicht immer zwingend hintereinander abgelegt werden müssen. Wie und wo diese Datenblöcke abgelegt werden, hängt von der Art der Speicherung ab. Für jede Art der Speicherung wird ein Inhaltsverzeichnis benötigt. Jede Art der Speicherung hat seine eigenen Vor- und Nachteile.

Kontinuierliche Speicherung

Key Takeaways

- Bei der kontinuierlichen Speicherung werden Dateien nebeneinander auf der Partition abgelegt.
- Der Beginn einer Datei und die Blocklänge werden in einer Inhaltstabelle abgelegt.
- Beim Löschen entstehen Löcher, welche nicht einfach wieder genutzt werden können.
- Extents ermöglichen es, Dateien auf mehrere Orte im Speicher aufzuteilen
- Die Runlists können sehr lang und speicherintensiv werden

Bei der kontinuierlichen Speicherung wird, wie der Name vermuten lässt, versucht, Dateien kontinuierlich, also hintereinander, abzulegen. Stellt man sich den Nutzdatenspeicher wie ein Array aus Speicherblöcken vor, so würde eine kontinuierliche Speicherung folgendermaßen aussehen:



Damit das System nicht immer alle Datenblöcke durchsuchen muss, nur um eine Datei zu finden, werden der **Beginn** und die **Blockanzahl** im Inhaltsverzeichnis, einer einfachen Tabelle gespeichert.

Diese Speichermethode hat den Vorteil, sehr simpel zu sein. Allerdings bringt sie einige Probleme mit sich.

Veränderung der Dateilänge:

Dateien können im Laufe der Zeit, durch Veränderung durch den User, wachsen oder schrumpfen. Die Dateien haben in dieser Speicherung aber keinen Platz um zu wachsen, da die folgenden Blöcke bereits von einer weiteren Datei belegt sind. Daher bleiben nur zwei Möglichkeiten:

1. Die Datei wird gelöscht und erneut an einen passenden Platz gespeichert
2. Die Datei wird in mehrere Datenblöcke aufgeteilt.

Ersteres ist vor allem bei großen Dateien sehr aufwendig und ineffizient. Außerdem entstehen dadurch wieder Lücken in der eigentlich kontinuierlichen Speicherung, und alle Dateien zu verschieben ist keine Alternative.

Letzteres ist möglich und wurde auch gemacht. Das Konzept nennt sich **Extents**.

Wachstum des Inhaltsverzeichnisses

Gerade bei Systemen, welche viele kleine Dateien speichern müssen, kann das Inhaltsverzeichnis einen massiven Speicherplatzverbrauch bedeuten, da für jede Datei ein neuer Eintrag angelegt werden muss. Extents verschlimmern dieses Problem noch einmal deutlich, da sie mehrere Einträge pro Datei brauchen.

Kontinuierliche Speicherung mit Extents

Bei dieser Unterart der kontinuierlichen Speicherung hat das Inhaltsverzeichnis nicht nur einen Eintrag pro Datei, sondern potentiell mehrere. Dateien werden, wenn nötig, an

verschiedenen Orten auf der Festplatte gespeichert. Das ermöglicht das Ausnutzen von bei der Dateilöschung entstehenden Blocklücken.

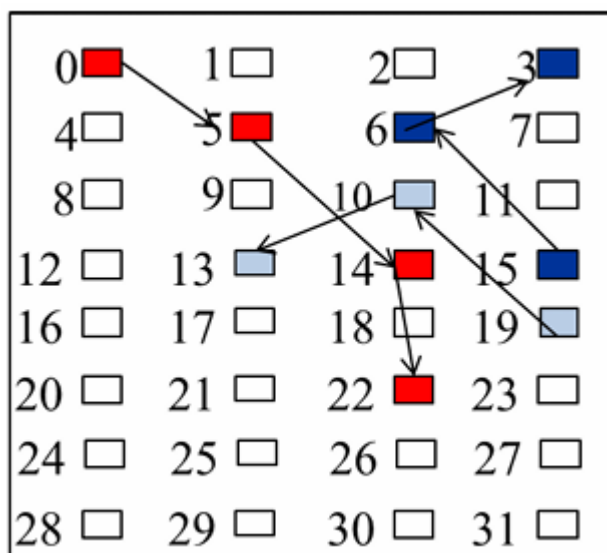
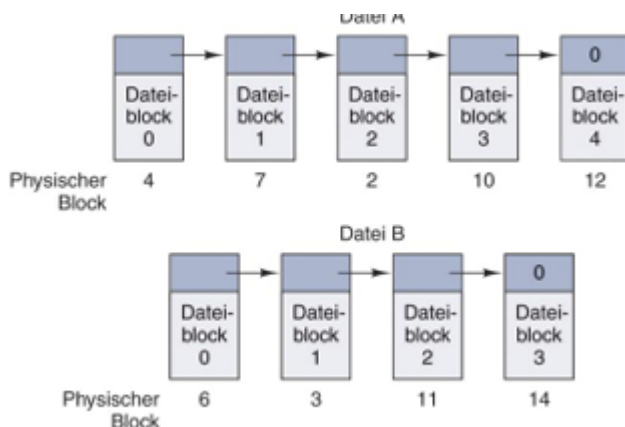
Diese Technik hat den gravierenden Nachteil, dass das Inhaltsverzeichnis dadurch stark anwächst, da für jeden Extent ebenso die Startposition und Länge gespeichert werden muss. Gerade für Systeme mit vielen Dateien kann das Inhaltsverzeichnis viel Speicherplatz einnehmen.

Die Liste aller Extents einer Datei wird als **Runlist** bezeichnet.

Dieses System funktioniert gut, wenn selten Dateien gespeichert oder geändert werden, wie es bei manchen Medien wie CDs und DVDs der Fall ist. Gerade für HDDs ist dieses Verfahren ungeeignet, da der Lesekopf für jeden Extent potentiell bewegt werden muss.

Verkettete Speicherung

Die verkettete Speicherung nutzt eine verkettete Liste, welche die Speicherblöcke und ihre Nachfolger referenziert. Dadurch muss im Inhaltsverzeichnis immer nur ein Eintrag angelegt werden, da jeder Listeneintrag seinen unmittelbaren Nachfolger kennt, egal wo er sich physisch befindet.



Diese Speichermethode ist sehr flexibel und erlaubt quasi freies Speichern auf der gesamten Festplatte, allerdings hat sie gravierende Nachteile:

Geschwindigkeit

Aufgrund der Logik einer verketteten Liste muss das System bei der Suche von spezifischen Daten in einer Datei gezwungenermaßen alle Daten der Datei durchgehen, da es nicht wissen kann, wo die Daten wirklich liegen (**bad random access**)

Nicht geeignet für HDD, aus den gleichen Gründen wie Extents.

Verlustgefahr

Wie weiter oben erläutert können Speicherblöcke mit der Zeit kaputtgehen. Passiert dies mit einem Block in der Liste, ist der gesamte Rest der Liste automatisch verloren, wenn es keine anderen Sicherheitsmechanismen gibt.

FATx (File Allocation Table) - Reale Anwendung verketteter Speicherung

Key Takeaways

- FAT nutzt eine verkettete Speicherung mit einer zusätzlichen Tabelle, dem File Allocation Table
- Dieser liegt mehrfach im Speicher und speichert den Folgeblock eines jeden Blockes ab
- Dies erhöht die Sicherheit erheblich, kostet dafür RAM Speicher und kann bei Unterbrechungen zu Fehlern führen
- Die Zahl neben FAT (zB FAT16) ist die Bitanzahl der Speicheradressen
- Um den Speicherplatz zu verdoppeln, muss man entweder ein Bit mehr in der Adressierung verwenden (zB FAT16 -> FAT17) oder die Blockgröße verdoppeln

Das Dateisystem FATx nutzt das Prinzip der verketteten Speicherung, erweitert um den namensgebenden **File Allocation Table**. Diese Tabelle, welche zur Sicherheit mehrfach abgespeichert wird, übernimmt quasi die Zuweisung der Knoten. Man kann sich die FAT vorstellen wie eine Blockmap, welche für jeden Datenblock die Adresse des folgenden Datenblocks speichert. Das Inhaltsverzeichnis des Dateisystems muss hier, wie bei der standardmäßigen verketteten Liste, nur den Einstiegspunkt für jede Datei speichern.

Die Zahlen, welche bei der Bezeichnung angegeben sind (zB. FAT8, FAT16, FAT32, FAT64) stehen für die **Bitanzahl der Speicheradressen**. Je höher die Zahl, desto mehr Blockadressen können verwaltet werden, und umso kleiner kann ein einzelner Block sein.

Die FAT erhöht die Sicherheit erheblich, da die Nachfolger eines Blocks nicht mehr in ebenjenem gespeichert werden, sondern in der FAT abgebildet sind. Da die FAT in den Arbeitsspeicher geladen wird, profitiert sie von den sehr hohen Zugriffsgeschwindigkeiten.

Allerdings wird dadurch auch ein Teil des RAMs dauerhaft belegt.

Änderungen in den Dateien werden periodisch in die FAT aufgenommen. Dies passiert sehr oft, mehrmals pro Sekunde. Wird dieser Prozess jedoch unterbrochen, kann es zu Fehlern in der FAT und dadurch zu verlorenen Daten kommen.

Das ist auch der Grund, warum man USBs erst auswerfen sollte, da diese mit FAT arbeiten und durch das Auswerfen sichergestellt wird, dass alle Daten korrekt übertragen wurden.

Rechenbeispiel zum Zusammenhang von Blockgrößen und FAT Adressräumen

Angabe: Die Größe eines Blocks sei 32 KBs (beliebig gewählt)

Angabe: Das Dateisystem sei FAT16

Daraus folgt:

FAT16 $\rightarrow 2^{16}$ Speicheradressen.

2^{16} Adressen $\times$ 32KB pro Block = 2 097 152KB $\rightarrow$ 2048MB = 2GB maximal
adressierbarer Speicher

Bei stattdessen FAT32:

2^{32} Adressen $\times$ 32KB pro Block = 137 438 953 472KB $\rightarrow$ 134 217 728MB $\rightarrow$ 131 072GB $\rightarrow$ 128TB

Je größer die Blockgröße, desto mehr Speicher lässt sich mit weniger Adressen ansprechen: sei bei FAT16 die Blockgröße 64KB

2^{16} Adressen $\times$ 64KB pro Block $\rightarrow$ 4 194 304KB $\rightarrow$ 4096MB $\rightarrow$ 4GB maximal

Wäre es theoretisch FAT17

2^{17} Adressen $\times$ 64KB pro Block $\rightarrow$ 8 388 608KB $\rightarrow$ 8192MB $\rightarrow$ 8GB maximal

Will man also den Speicher verdoppeln, muss man entweder die Blockgröße verdoppeln oder ein Bit mehr für den Adressraum aufgeben.

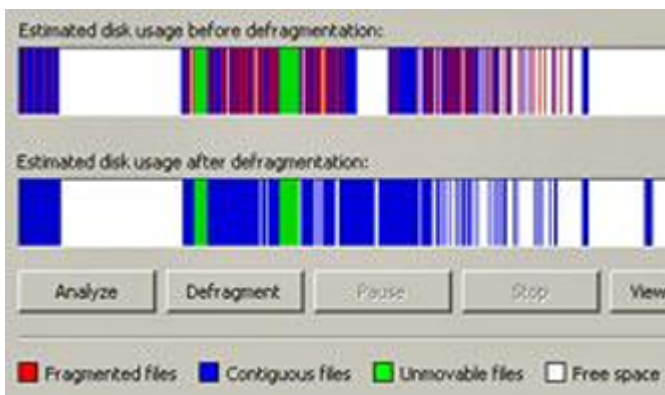
Defragmentierung

Key Takeaways

- Defragmentierung bezeichnet den Vorgang, Dateien, welche an unterschiedlichen Speicherstellen verstreut abgelegt sind, wieder physisch hintereinander abzulegen.
- Diese Technik wird vor allem für HDDs gebraucht

Fragmentierung beschreibt die Verteilung von einzelnen Dateien über viele Orte im Speicher, wie es zB bei der verketteten Speicherung gewollter und bei der kontinuierlichen Speicherung ungewollt passiert. Die Daten sind nicht alle nebeneinander, sondern an vielen Orten im Speicher verteilt.

Dies ist insbesondere für HDDs, welche ihren Lesekopf physisch bewegen müssen, sehr ineffizient. Daher gibt es Programme, welche die verschiedenen Dateiteile wieder hintereinander in den Speicher bringen. Dieser Vorgang heißt **Defragmentierung**



Indizierte Speicherung

Key Takeaways

- Indizierte Speicherung nutzt INodes, welche immer im ersten Block einer Datei liegen
- INodes halten die Abfolge der LBAs in sich
- Der Vorteil ist, dass nicht alle Informationen für jeden Block immer im RAM sein müssen
- Nachteil ist, dass es wieder verlustanfällig ist, da ein INode Block schonmal kaputt gehen könnte.

Die indizierte Speicherung greift das Prinzip der verketteten Speicherung mit FAT auf, allerdings mit einer großen Änderung:

Anstatt einer großen Tabelle wie dem FAT werden hier sogenannte **INodes** verwendet. INodes sind quasi ein Feld, welches immer im ersten Block einer Datei abgelegt ist. Um diesen ersten Block zu finden wird eine Inhaltstabelle verwendet.

Die INodes hält die Adresse aller Blöcke dieser Datei. Das bedeutet, dass beim Aufruf einer Datei erst in der Inhaltstabelle nach dem ersten Block gesucht wird, und danach in der INode alle Datenblöcke zu finden sind.

Dieses System hat den großen Vorteil, dass nicht zu jeder Zeit alle Blockadressen und Folgeadressen gleichzeitig im RAM geladen werden müssen, was die Speicherkosten massiv senkt.

Da diese INodes aber nur in je einem Block abgelegt sind, ist die Datei verloren, sollte dieser Block kaputt gehen. Dies ist allerdings viel unwahrscheinlicher als bei der reinen verketteten Speicherung, wo jeder Block einer Datei potentiell zu Datenverlust führt.

Verzeichnisse

Key Takeaways

- Verzeichnisse sind (in LINUX UNIX) als Dateien implementiert
- Sie beinhalten den Verzeichnisnamen, sowie eine Referenz auf alle Dateien im Verzeichnis und deren Attribute
- Ein Dateiname in einem Verzeichnis hat entweder eine fixe Maximallänge oder braucht einen Arbeitsspeicher (Heap) um variabel zu sein

Technisch gesehen ist ein Verzeichnis (zmnst unter LINUX / UNIX) ebenfalls nur eine Datei. Verzeichnisse haben den Auftrag, Dateien zu logisch zusammengehörenden Gruppen zusammenzufassen.

In Verzeichnissen selbst ist der Verzeichnisname gespeichert, welcher Teil der Verzeichnisadresse ist, welches das OS wiederum nutzt um Dateien zu finden (siehe File Explorer)

Weiters liegt in einem Verzeichnis für jede Datei welche es beinhaltet eine Referenz, welche je nach Speicherung auf die Inode oder den Startblock der Datei zeigt. Ebenso hält ein Verzeichnis die Attribute jeder Datei.

Hierbei ist vor allem der Dateiname hervorzuheben. Da auch Verzeichnisse auf Datenblöcken gespeichert werden müssen, muss entschieden werden, wie mit langen Dateinamen umgegangen wird:

1. Es gibt eine feste Länge / feste Menge Speicherplatz pro Dateiname
2. Es gibt einen Heap auf welchen alle Dateinamen abgelegt werden können

Ersteres ist sehr simpel, aber natürlich einschränkend

Zweiteres ist aufwendiger, sowohl für Speicher als auch Geschwindigkeit, erlaubt aber Namen "jeglicher" Länge.

Beispiele von konkreten Dateisystemen

Key Takeaways

- Linux ext4 verwendet indizierte Speicherung mit Extents und Journaling
- Auf einer Partition gibt es verschiedene wichtige Bereiche, wie den Superblock, Bootblock, Freispeicherverw, Inodes und Datenblöcke
- Blöcke in einer Partition werden nach ihren Aufgaben gruppiert, was vor allem für schnelleres Lesen effizient ist.

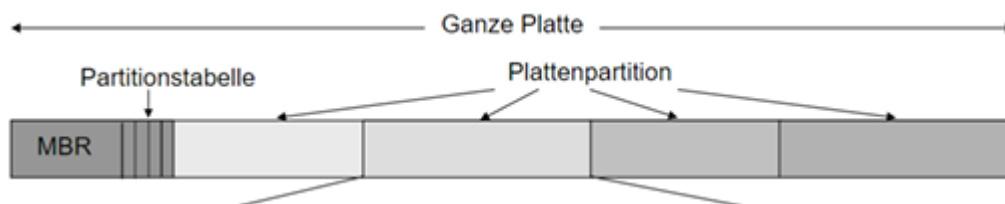
Linux ext4

ext steht für Extended Dateisystem und ist der Nachfolger von ext 3 und 2, welche um 16 Bit für die Adressierung von Datenblöcken, Extents und **Journaling** erweitert wurden.

Prinzipiell verwendet ext4 eine indizierte Speicherung, also INodes.

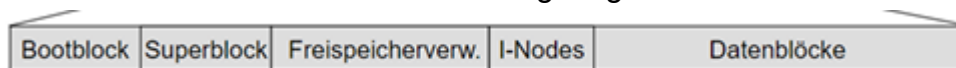
Aufbau von Platte, Datei und

Unter Linux gilt folgender Aufbau einer gesamten Festplatte:



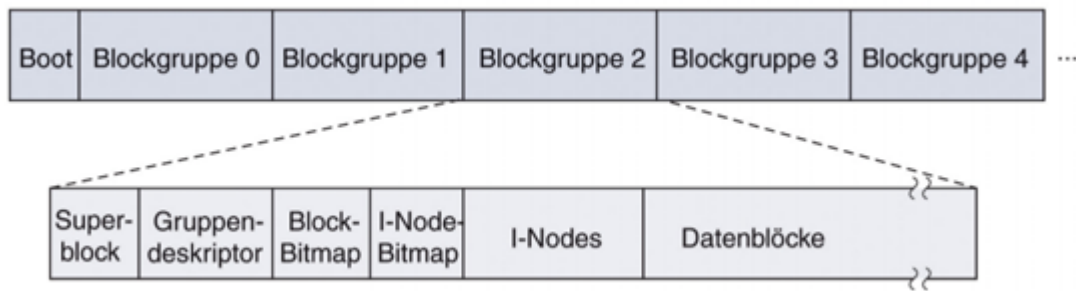
Wobei ganz vorne der MBR (Master Boot Record) und die Partitionstabelle liegt. Danach kommen die einzelnen Partitionen. Dieser Aufbau ist sehr standardmäßig und in dieser Form noch nicht von Linux abhängig sondern von BIOS.

Eine nach ext4 formatierte Partition folgt folgendem Aufbau:



- Bootblock
 - Der Bootblock enthält Informationen für den Bootvorgang bzw einen Bootloader
- Superblock
 - Wichtige Parameter für das Dateisystem:
 - Name
 - Schutzmarkierungen
 - ...
- Freispeicherverwaltung
 - siehe Kapitel oben
- INodes
 - Speicherposition aller INodes für die folgenden Dateien und Verzeichnisse
- Nutzdatenblöcke
 - Alle real mit Nutzdaten gefüllten Speicherblöcke

Innerhalb der Partition findet wiederum eine Unterteilung in Blöcke statt. Hier sind nicht die einzelnen Speicherblöcke, sondern Blockgruppen gemeint, welche wiederum eigene Aufgaben erfüllen:



- Superblock
 - Speichert Informationen über das Layout des Dateisystems, wie die Anzahl der INodes und Blöcke, sowie Zeiger auf die Liste der Freispeicherverwaltung
- Gruppendeskriptor
 - Enthält Informationen über die Position von BitMaps, INodes, freie Blöcke und die Verzeichnisanzahl.
- Dateien und Verzeichnisse - werden für höhere Datenlokalität an einem Ort gespeichert (Gut für HDDs aber auch für SSDs, nebeneinander zuzugreifen ist einfach schneller)

INodes unter Linux

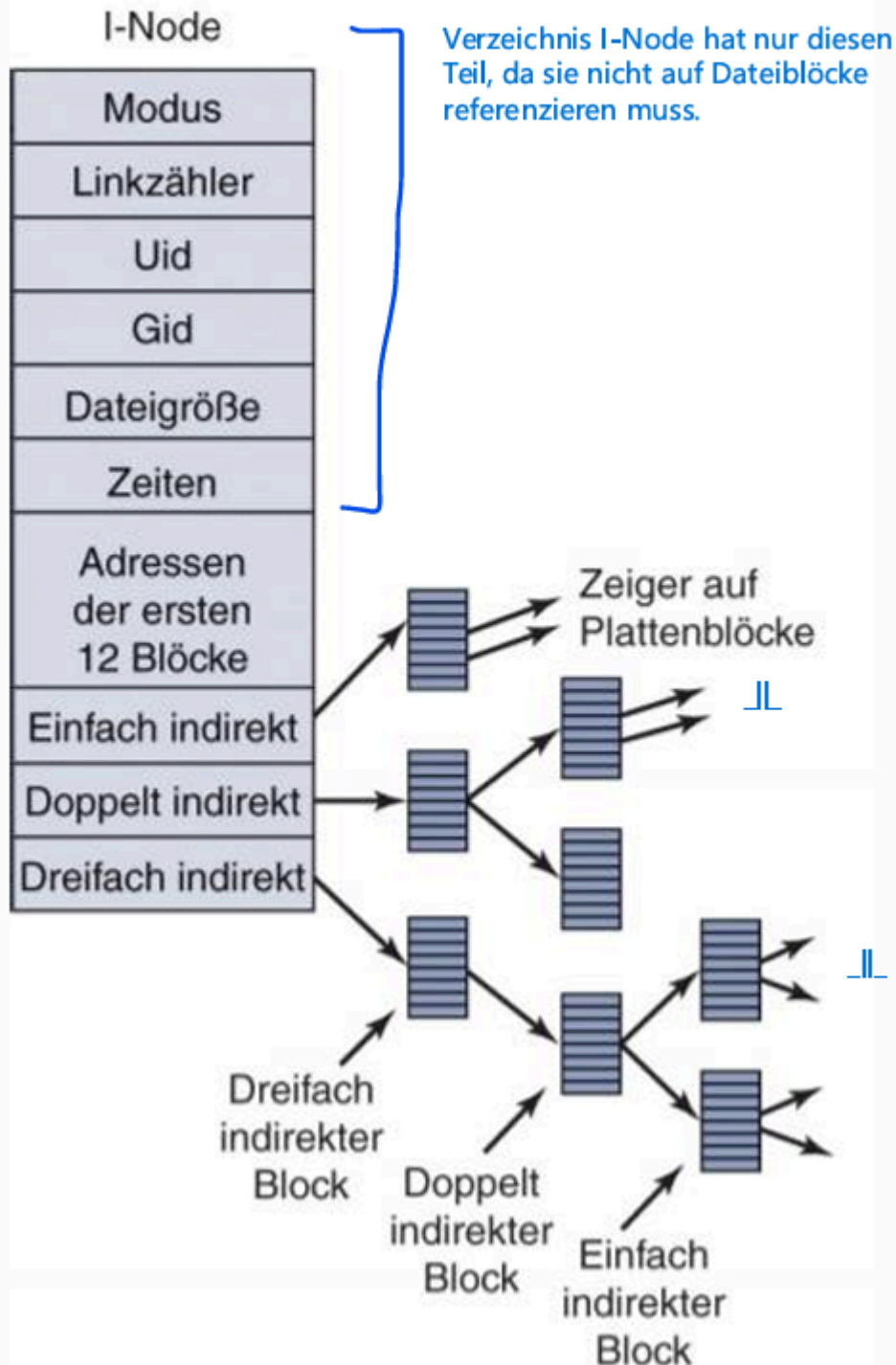
Key Takeaways

- INodes sind Datenstrukturen mit Metadaten und Referenzen auf andere Blöcke
- INodes referenzieren Datenblöcke entweder direkt oder indirekt über Blöcke welche ebenfalls Referenzen enthalten.
- INodes unterstützen biszu 3-Fache indirekte Referenzierung
- Die maximale Größe einer Datei hängt von der Referenzanzahl und Blockgröße ab (siehe Bsp.)

Unter ext4 werden INodes in einer INode List abgespeichert. Jedes Objekt (Datei und Verzeichnis) hat eine eigene INode. Diese INodes sind ähnlich aufgebaut wie oben beschrieben, haben allerdings zusätzlich zu den Metainfos und LBAs noch Platz für weitere Zeiger. Diese zeigen auf **indirekte, doppelt indirekte, und triple indirekte Blöcke**

Das bedeutet, dass nicht direkt auf einen Datenspeicherblock referenziert wird, sondern auf eine weitere Node, welche wiederum Zeiger auf Plattenblöcke enthält. Dieses Prinzip tritt bis zu 3 Mal nacheinander auf (deshalb auch **triple** indirekt)

Die indirekt referenzierten Blöcke sind **KEINE INodes**. Sie sind einfache Datenblöcke, welche vollständig für die Referenzspeicherung verwendet werden. Genauerer hierzu siehe im Beispiel unten:



Beispiel: Maximale Filegröße unter Linux ext4

Angabe:

Größe eines Datenblocks: 1KB

Adresslänge einer LBA: 32Bit

$(1024 * 8) \text{ Bit} / 32 \text{ Bit} = 256$ (Anzahl an Adressen pro Block)

Direkte Adressierung:

Eine INode hat 12 Zeiger -> maximal 12 Blöcke -> 12KB maximale Größe

Einfach indirekte Adressierung:

Zusätzlich kommt jetzt ein indirekter Zeiger auf einen vollen Adressblock hinzu

$$12 * 1KB + 256 * 1KB = 12KB + 256KB = 268KB \text{ maximale Größe}$$

Doppelte indirekte Adressierung:

Nun wird zusätzlich ein doppelt adressierender Block verwendet:

$$268KB + 256 \text{ indirekte Adressen} * 256 * 1KB = 65\,804KB = \sim 64,3MB$$

Dreifach indirekte Adressierung:

Nun wird zusätzlich ein dreifach adressierender Block verwendet:

$$65\,804 + 256 \text{ indirekte Adr.} * 256 \text{ Indirekte Adr} * 256 * 1KB = 16\,843\,020KB = 16\,448,3MB = 16,06 \text{ GB maximale Größe}$$

Beispiel Teil 2: Andere Blockgröße:

Angabe:

Größe eines Datenblocks: 4KB

Adresslänge einer LBA: 32Bit

$$(4096 * 8) \text{ Bit} / 32 \text{ Bit} = 1024 \text{ (Anzahl an Adressen pro Block)}$$

Direkte Adressierung:

Eine INode hat 12 Zeiger -> maximal 12 Blöcke -> 48KB maximale Größe

Einfach indirekte Adressierung:

Zusätzlich kommt jetzt ein indirekter Zeiger auf einen vollen Adressblock hinzu

$$12 * 4KB + 1024 * 4KB = 48KB + 4096KB = 4144KB \text{ maximale Größe}$$

Doppelte indirekte Adressierung:

Nun wird zusätzlich ein doppelt adressierender Block verwendet:

$$4144KB + 1024 \text{ indirekte Adressen} * 1024 * 4KB = 4\,198\,448KB = 4\,100MB = 4TB$$

Dreifach indirekte Adressierung:

Nun wird zusätzlich ein dreifach adressierender Block verwendet:

$$4\,198\,448KB + 1024 \text{ indirekte Adr.} * 1024 \text{ Indirekte Adr} * 1024 * 4KB = 4\,299\,165\,744KB = 4\,198\,404MB = 4\,100 \text{ GB} = 4TB \text{ maximale Größe (groß)}$$

ext4 Verzeichnisse

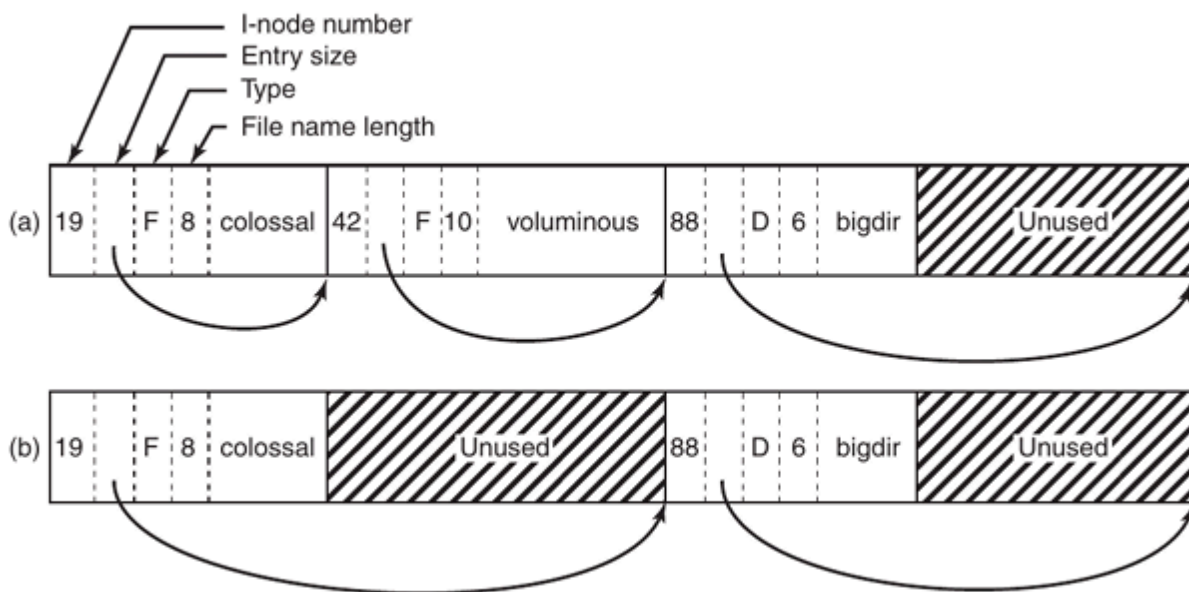
✎ Key Takeaways

- Verzeichnisse in Linux sind ebenfalls Dateien. Sie sind eine verkettete Liste.
- Verzeichnisse enthalten Metadaten über das Verzeichnis und Referenzen auf die Dateien des Verzeichnisses

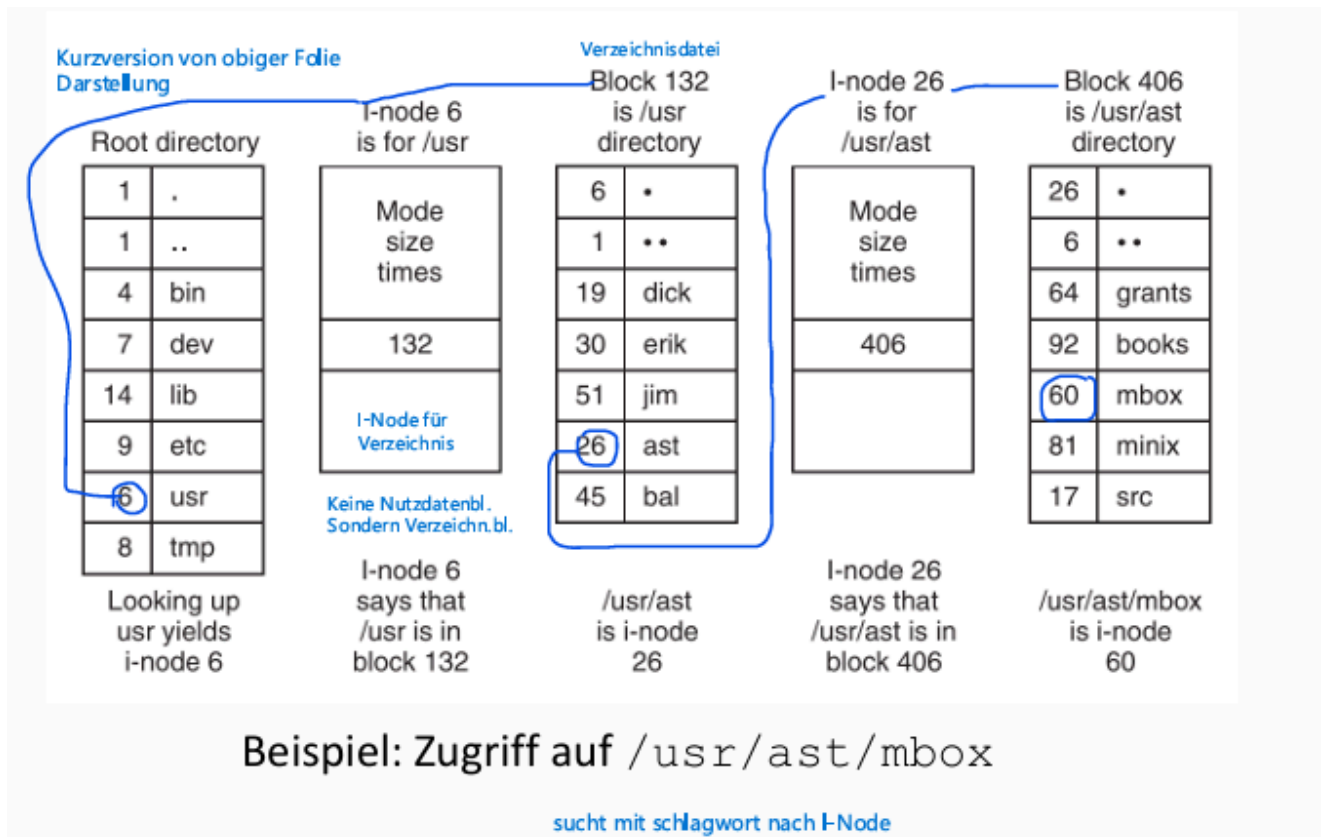
Ein Verzeichnis wird in ext4 als verkettete Liste dargestellt. Diese Liste besteht aus sogenannten **directory entries**, welche im wesentlichen aus den Metadaten der Datei bestehen. Hierbei sind folgende relevant:

- Inode Nummer des Objekts (Hier Objekt, da ein Verzeichnis auch ein Verzeichnis beinhalten kann)
- Größe des Directory entries (abhängig von der Länge des Dateinamens)
- Type (F -> File, D -> Directory)
- File name length (nötig um die Größe zu berechnen)

Hier ist eine Skizze zweier Verzeichnisse. Die Grenzen nach den großen Datenblöcken sind der Beginn des nächsten directory entries:



Dateizugriff über eine Pfadangabe:



Erweiterte Funktionalitäten eines Dateisystems:

Zusätzlich zu den oben erläuterten Standardfunktionalitäten wurden Dateisysteme über die Jahre um einige Zusatzfunktionalitäten erweitert, welche primär zur Datensicherung oder Verschnellerung eines oder mehrerer Prozesse im Datenmanagement gedacht sind. Die wichtigsten werden unten genauer erläutert:

Journaling

Key Takeaways

- Journaling versucht, Daten trotz eines Absturzes konsistent zu halten
- Hierzu werden alle vorzunehmenden Änderungen bei einem Prozess in einem LOG abgelegt
- Das Transaktionsende gibt an, ob der LOG vollständig ist.
- Stürzt das System ab, wird nach den LOGs gesucht, ist ein Transaktionsende vorhanden, werden fehlende Operationen ausgeführt, fehlt es, werden alle protokollierten Operationen rückgängig gemacht
- 2 Arten: Full- und Meta-Journaling, wobei Full sehr langsam ist aber Nutzdaten sichert.

Journaling ist eine Technik um Daten nach einem Computerabsturz automatisch auf Konsistenz zu prüfen und unfertige Prozesse zu erkennen und diese abzubrechen bzw. fertigzustellen.

Hierfür werden am Beginn eines Prozesses (zB dem Speichern einer Datei) LOG-Files angelegt, welche alle nötigen Schritte zum Abschluss dieses Prozesses beinhalten.

Wird der Prozess vollständig abgearbeitet, wird das LOG-File als letzter Schritt wieder gelöscht. Passiert jetzt ein Absturz, wird das durch das oben erläuterte Dirty Bit erkannt und das OS beginnt damit, diese LOG-Files zu suchen. Dann wird nach dem **Transaktionsende** gesucht. Dieses gibt an, ob alle Operationen vollständig im LOG-File geschrieben wurden, da es immer der letzte Eintrag im LOG-File sein muss. Ist das Transaktionsende nicht vorhanden, werden alle Änderungen rückgängig gemacht. Ist es vorhanden, werden die Änderungen fertig ausgeführt.

Hierbei unterscheidet man zwischen **Full-Journaling** und **Meta-Journaling**, wobei ersteres die Konsistenz aller Meta- und Nutzdaten gewährleistet (und dafür einen hohen Preis von der Geschwindigkeit fordert) und letzteres um einiges performanter nur die Metadaten konsistent hält.

Extents

Ähnlich wie bei der kontinuierlichen Speicherung ist ein Extent eine Referenz auf mehrere aufeinanderfolgende Blöcke (max. 128MB bei ext4). Diese werden über die INodes anstatt eines einzelnen Blockes referenziert. Bis zu 4 Extents können direkt referenziert werden, für alles weitere wird eine Baumstruktur verwendet.

Diese Technik hat vor allem für große Dateien Vorteile, da sie die Fragmentierung minimiert und weniger indirekte Referenzen über INodes braucht. Die meisten modernen Betriebssysteme verwenden Extents.

Copy on Write (COW) und B+ Trees

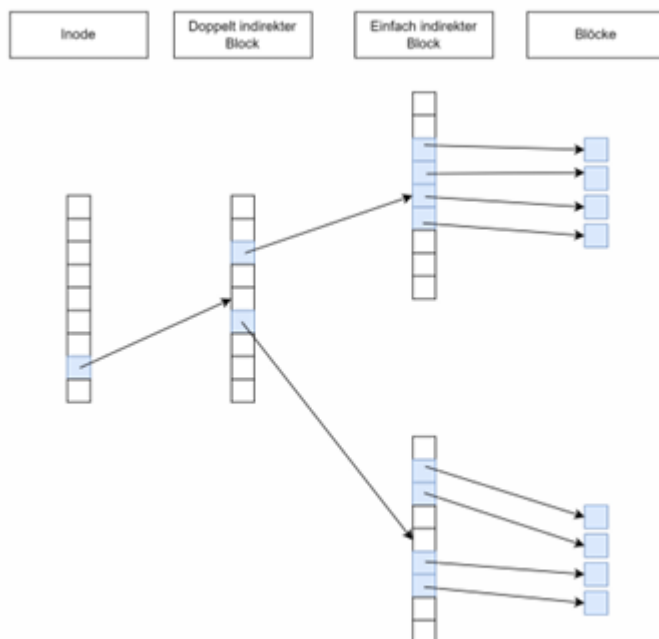
Key Takeaways

- COW ermöglicht Sicherung und Versionierung von Änderungen (inkrementell)
- Bei Leseprozessen passiert nichts
- Bei Schreibprozessen werden alle Referenzen kopiert, außer jene auf die geänderten Blöcke
- Durch doppeltreferenzierung entsteht inkrementelle Sicherung der Änderungen (Versionierung)

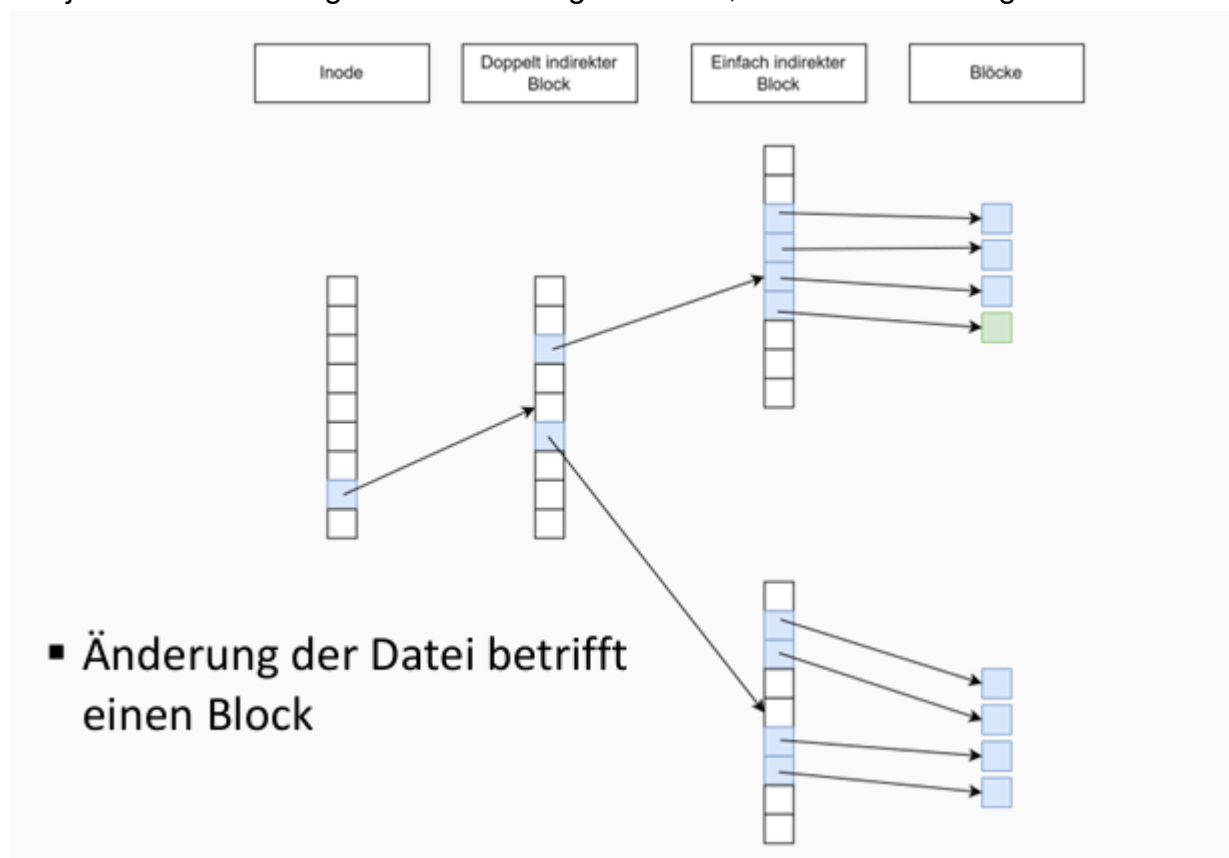
COW ist eine andere Idee, wie man Daten sicher ändern kann ohne Verluste bei Crashes zu riskieren. COW hat den Vorteil, dass sowohl Nutz- als auch Metadaten gesichert sind, ohne große Performanceeinbußen zu haben und ist auch für **inkrementelle** Sicherungen gut.

COW, wie der Name schon sagt, Kopiert Daten erst, wenn sie geändert (geschrieben) werden. Standardmäßig (für Leseoperationen) verwendet COW das oben erläuterte

standardmäßige Inode Prozedere:



Wir jetzt eine Änderung in der Datei vorgenommen, also zB ein Block geändert:

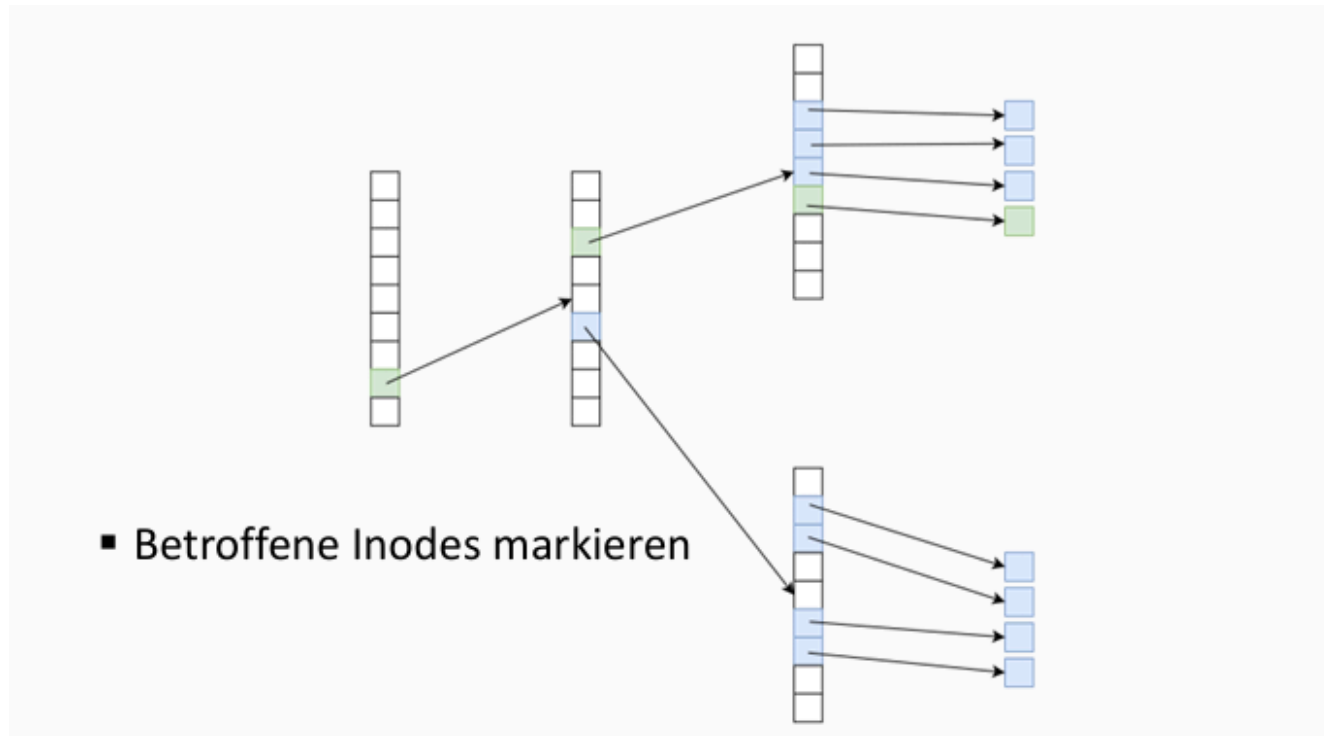


Dann passiert folgendes:

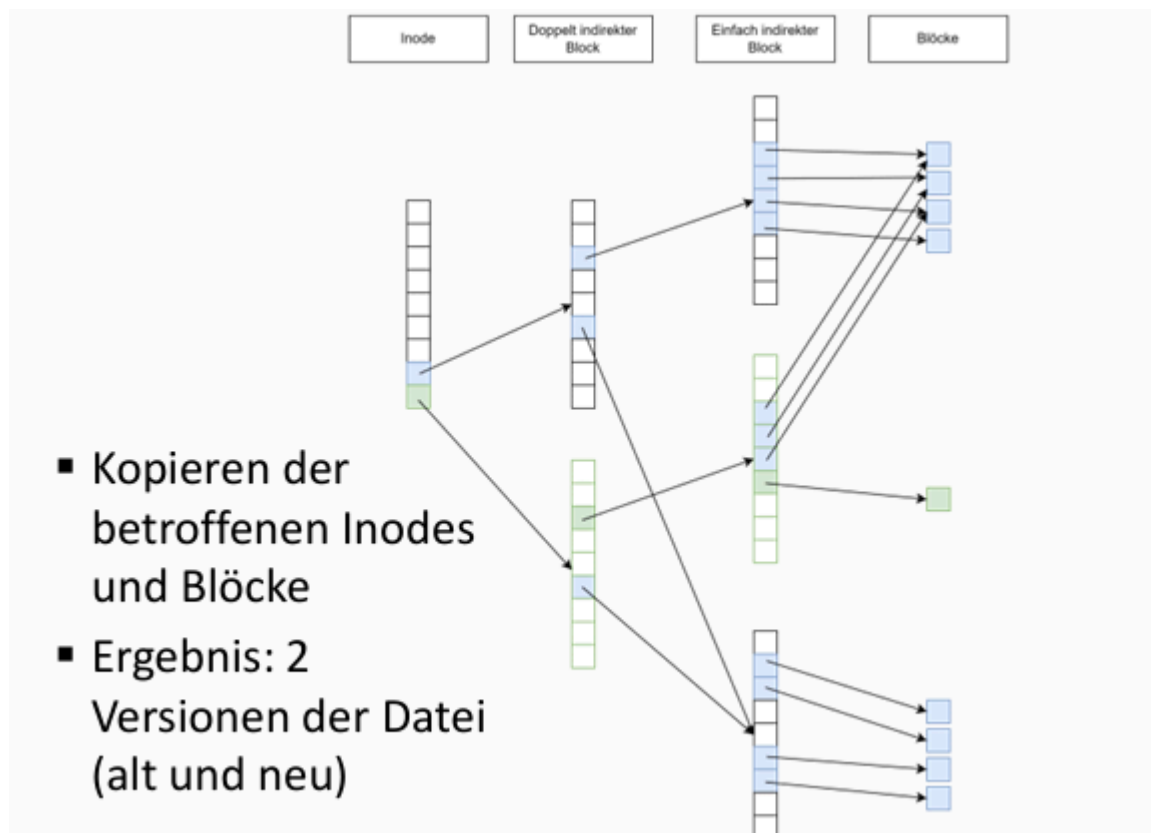
- Eine neue Referenz wird in der Inode angelegt. Diese referenziert auf eine Kopie der indirekten Blöcke. Jede diese Kopie beinhaltet **alle Referenzen** des Originals. Das bedeutet, dass am Ende für jeden Block 2 Zeiger existieren.

- Da COW Änderungen immer auf leere Blöcke schreibt, wird der geänderte Block **nur** von der Kopie referenziert, und das Original besteht weiter. Das bedeutet, dass es 2 Versionen der Datei gibt. Eine mit den Änderungen, und eine ohne. Es wurden aber nur die Änderungen neu gespeichert, und nicht die gesamte Datei, und beide sind zeitgleich am System verwendbar.

Betroffene originale Referenzen:



Hier sieht man die zweite Referenz in der INode, welche auf Kopien der Blöcke referenziert:



Alle Blöcke doppelt referenziert, außer die geänderten, diese jeweils einmal.

